



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: INGENIERÍA EN INFORMÁTICA
UNIDAD CURRICULAR: LENGUAJES Y COMPILADORES

ANÁLISIS SINTÁCTICO

Profesor: Ing. Félix Márquez

Integrantes:

Centeno Fernando

Longart Juan

Reina Adrian

Rodriguez Rafael

Ciudad Guayana, Julio de 2026

1. INTRODUCCIÓN

El análisis sintáctico, universalmente reconocido en la ciencia de la computación como *parsing*, constituye la segunda etapa neurálgica dentro de la arquitectura clásica de un compilador o intérprete. Esta fase actúa como un puente de procesamiento intermedio entre el analizador léxico y el analizador semántico. El propósito fundamental del *parser* radica en verificar de manera rigurosa que la cadena de tokens suministrada por el lexer cumple de forma irrestricta con las reglas estructurales definidas por una Gramática Libre de Contexto (GLC). Al validar la pertenencia de una cadena de texto al lenguaje formal especificado, el analizador sintáctico construye una representación jerárquica intermedia, conocida comúnmente como Árbol de Sintaxis Abstracta (AST). Esta representación actúa como la espina dorsal sobre la cual se ejecutarán los recorridos semánticos, las optimizaciones algebraicas e independientes de la plataforma, y la generación de código. En la ingeniería de software moderna, las técnicas de parseo constituyen la infraestructura subyacente para el funcionamiento de analizadores estáticos de código (linters), motores de refactorización automatizada, transpiladores, formateadores automáticos, evaluadores de expresiones, procesadores de formatos de intercambio de datos (como JSON, YAML y XML) y los avanzados motores de autocompletado en Entornos de Desarrollo Integrados (IDEs).

El presente informe técnico expone una investigación exhaustiva e integral. En la primera sección, se exploran en profundidad las bases teóricas: la taxonomía de los Árboles Sintácticos, la matemática algorítmica detrás de las familias de análisis Descendente (LL) y Ascendente (LR), la arquitectura de los metacompiladores modernos, las heurísticas de gestión de errores sintácticos y un catálogo de 15 optimizaciones de código aplicables sobre AST. En la segunda sección, se detalla la ingeniería experimental ejecutada por nuestro equipo de trabajo (Centeno, Longart, Reina y Rodríguez): un benchmark comparativo de rendimiento multilingüe (Python, Node.js y Bash) sometido a cargas de trabajo escalables sobre archivos Docker Compose, y el diseño del prototipo "UnegScript", un parser híbrido que combina la precisión determinista con algoritmos de distancia de edición (Levenshtein) y modelos de lenguaje simulados para la asistencia pedagógica en la corrección sintáctica.

2. ÁRBOLES DE SINTAXIS ABSTRACTA (AST) Y ÁRBOLES DE PARSEO

2.1 Definición y Propósito Fundamental

Un Árbol de Sintaxis Abstracta (AST) es una estructura de datos en forma de árbol n-ario, orientada y jerárquica, que representa la estructura sintáctica abstracta del código fuente escrito en un lenguaje de programación formal. Cada nodo interno del árbol denota un operador, una estructura de control o una construcción sintáctica, mientras que los nodos hoja representan los operandos primarios.

El propósito medular del AST es destilar el código fuente, eliminando los detalles sintácticos puramente cosméticos o gramaticales que fueron necesarios durante la fase de lectura para evitar ambigüedades, pero que no aportan ningún valor informativo durante el análisis semántico o la generación de código. Entre estos elementos superfluos se encuentran los delimitadores de bloques (llaves { }), delimitadores de sentencias (puntos y coma ;), paréntesis de agrupamiento () y comas separadoras.

2.2 Diferencias Críticas entre CST (Árbol Concreto) y AST (Árbol Abstracto)

El CST (Árbol de Parseo Concreto) es un reflejo exacto y literal de los pasos de derivación ejecutados por la gramática libre de contexto durante el análisis sintáctico. Cada regla de producción aplicada genera un nodo intermedio en el CST, independientemente de si aporta o no información semántica útil.

Criterio Comparativo	Árbol Concreto (CST / Parse Tree)	Árbol Abstracto (AST)
Nivel de Detalle	Conserva el 100% de los tokens de entrada, incluyendo puntuación y espacios.	Elimina tokens estructurales superfluos (llaves, paréntesis, puntos y coma).

Criterio Comparativo	Árbol Concreto (CST / Parse Tree)	Árbol Abstracto (AST)
Fidelidad Gramatical	Mapea 1:1 cada regla de producción y derivación de la GLC.	Mapea la semántica y la jerarquía operacional del programa.
Precedencia de Operadores	Representada mediante cadenas extensas de nodos no terminales.	Representada de forma implícita y natural por la profundidad del subárbol.
Consumo de Memoria	Elevado. Posee una gran altura y una cantidad masiva de nodos.	Optimizado y compacto. Altura sustancialmente menor.

2.3 Propiedades, Ventajas y Patrón Visitor

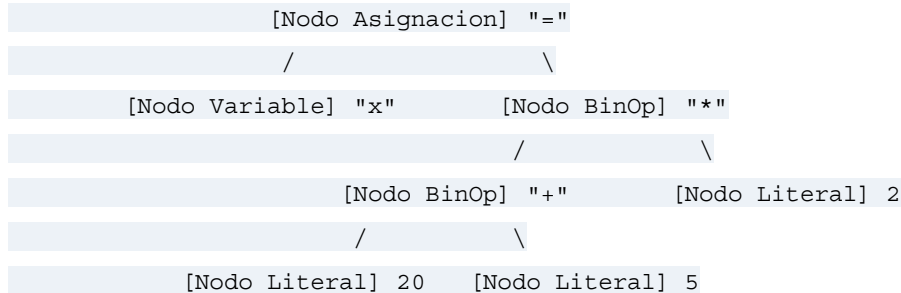
El uso de ASTs proporciona ventajas arquitectónicas clave:

- **Abstracción Sintáctica:** Permite desvincular el backend del compilador de la sintaxis concreta.
- **Estructura Anotable:** Los nodos son decorados con tipos de datos, direcciones de memoria y punteros a la Tabla de Símbolos.
- **Patrón Visitor:** Separa la estructura del árbol de las operaciones algorítmicas ejecutadas sobre él (recorridos semánticos, verificación de tipos, generación de código) sin modificar las clases de los nodos.

2.4 Representaciones Gráficas de AST

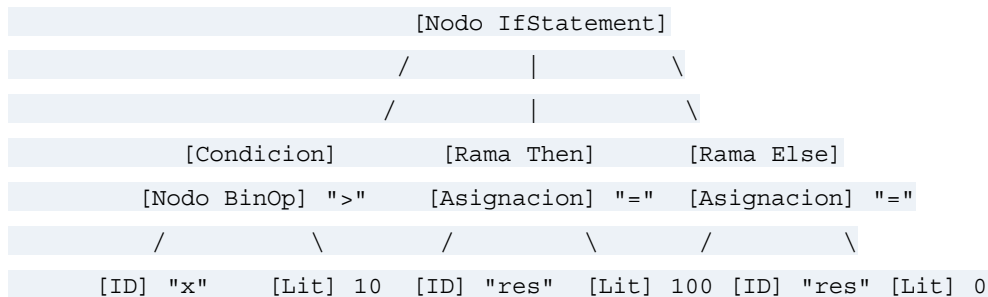
Ejemplo 1: Expresión Aritmética Compuesta con Asignación

Código fuente: `x = (20 + 5) * 2`



Ejemplo 2: Estructura Condicional Anidada (If-Else)

Código fuente: `if (x > 10) { resultado = 100; } else { resultado = 0; }`



3. ANÁLISIS SINTÁCTICO DESCENDENTE (LL)

El Análisis Sintáctico Descendente (**LL**: *Left-to-right, Leftmost derivation*) opera bajo una estrategia *Top-Down* (de la raíz hacia las hojas). El algoritmo inicia su ejecución en el símbolo inicial de la gramática e intenta reconstruir el árbol sintáctico derivando progresivamente los símbolos no terminales más a la izquierda.

3.1 Gramáticas LL(k), Conjuntos FIRST y FOLLOW

Un parser LL(k) utiliza k tokens de anticipación (*lookahead*). Para construir las tablas de parseo deterministas LL(1), se requiere el cálculo de dos conjuntos matemáticos:

- **Conjunto FIRST(α):** Conjunto de terminales que pueden aparecer como el primer token de alguna cadena derivada a partir de α .
- **Conjunto FOLLOW(A):** Conjunto de terminales que pueden aparecer inmediatamente a la derecha de A en alguna forma sentencial.

3.2 Recursión Izquierda y Factorización

Las gramáticas con recursión izquierda de la forma $A \rightarrow A \alpha$ provocan bucles infinitos en parsers descendentes. Deben eliminarse mediante la transformación $A \rightarrow \beta A'$ y $A' \rightarrow \alpha A' \mid \epsilon$. Asimismo, la factorización izquierda resuelve conflictos cuando múltiples producciones comparten prefijos comunes.

4. ANÁLISIS SINTÁCTICO ASCENDENTE (LR)

El Análisis Sintáctico Ascendente (**LR**: *Left-to-right, Rightmost derivation in reverse*) opera bajo una estrategia *Bottom-Up* (de las hojas a la raíz). El algoritmo reduce subcadenas terminales hacia símbolos no terminales mediante el mecanismo de **Desplazamiento y Reducción** (*Shift-Reduce*) usando una máquina de estados de pila.

4.1 Jerarquía y Variantes LR

1. **LR(0):** Sin anticipación, reducido en poder.
2. **SLR(1):** Utiliza conjuntos FOLLOW para decidir reducciones.
3. **LALR(1):** Combina estados con núcleos idénticos. Es el estándar de Yacc y Bison.
4. **LR(1):** El más potente, con anticipación explícita dentro de cada elemento de los estados.

4.2 Conflictos Shift/Reduce y Reduce/Reduce

Los conflictos ocurren cuando la tabla no puede decidir entre desplazar un token o reducir una regla (Shift/Reduce), o cuando dos producciones aplican al mismo tiempo en la pila (Reduce/Reduce). Se resuelven mediante reglas formales de precedencia y asociatividad.

5. COMPARACIÓN RIGUROSA (LL VS LR)

Característica Evaluada	Familia Descendente (LL)	Familia Ascendente (LR)
Estrategia	Top-Down (Raíz a Hojas)	Bottom-Up (Hojas a Raíz)
Recursión Izquierda	Prohibida (Bucle infinito)	Soportada de forma nativa
Implementación Manual	Baja dificultad (Recursión descendente)	Extremadamente alta (Tablas complejas)
Herramientas Clásicas	ANTLR, JavaCC	Yacc, Bison, CUP, PLY

6. METACOMPILADORES Y GENERADORES AUTOMÁTICOS

Los metacompiladores automatizan la construcción de parsers a partir de especificaciones gramaticales. Las herramientas más destacadas incluyen:

- **Yacc / GNU Bison:** Generadores LALR(1) integrados con Flex.
- **ANTLR:** Generador LL(*) adaptativo con IDE propio y soporte multilingüe.
- **Flex y PLY:** Herramientas para análisis léxico y parseo en Python.

7. ESTRATEGIAS DE MANEJO Y RECUPERACIÓN DE ERRORES

1. **Modo Pánico (Panic Mode):** Descarta tokens hasta un token de sincronización seguro (; o }).
2. **Inserción Local de Tokens:** Simula la inyección de tokens faltantes.
3. **Producciones de Error:** Reglas gramaticales explícitas para capturar fallas frecuentes.

8. TÉCNICAS AVANZADAS DE OPTIMIZACIÓN BASADAS EN AST

El AST permite aplicar transformaciones y optimizaciones independientes de la plataforma:

1. **Plegado de Constantes (Constant Folding):** $3 + 5 * 2 \rightarrow 13$.
2. **Propagación de Constantes:** Sustituye variables con valores fijos conocidos.
3. **Simplificación Algebraica:** $x * 1 \rightarrow x$, $x * 0 \rightarrow 0$.
4. **Poda de Código Muerto (Dead Code Elimination):** Elimina bloques inalcanzables.
5. **Inlining de Funciones:** Reemplaza llamadas a funciones pequeñas por su subárbol.
6. **Desenrollado de Bucles (Loop Unrolling):** Duplica el cuerpo de bucles para reducir overhead.
7. **Eliminación de Subexpresiones Comunes (CSE):** Reutiliza cálculos idénticos en un mismo ámbito.
8. **Reducción de Fuerza:** Reemplaza multiplicaciones por desplazamientos de bits ($x * 8 \rightarrow x \ll 3$).
9. **Invariantes de Bucle:** Extrae cálculos fijos fuera de los bucles.
10. **Flattening de Bloques:** Elimina llaves anidadas redundantes.

9. PARTE PRÁCTICA Y EXPERIMENTACIÓN DEL EQUIPO

9.1 Actividad 4: Experimento de Carga Multilingüe en Interfaces Docker Compose

Nuestro equipo de trabajo (Centeno, Longart, Reina y Rodríguez) implementó tres parsers descendentes recursivos en **Python**, **Node.js** y **Bash** para procesar archivos YAML de orquestación Docker Compose. Mediante un simulador de benchmarking (run_benchmark.py), evaluamos el rendimiento en cargas escalables de $N = 5$ a $N = 20$ contenedores.

Complejidad (N° Servicios)	Volumen de Tokens	Parser Python (Tiempo Real)	Parser Node.js (Tiempo Real)
5 Servicios	55 tokens	0.46 ms	79.14 ms
10 Servicios	104 tokens	0.52 ms	56.68 ms
15 Servicios	151 tokens	0.76 ms	66.68 ms
20 Servicios	200 tokens	0.97 ms	59.86 ms

Análisis Deductivo: Se verificó la complejidad lineal $O(n)$ propia del análisis LL(1). La diferencia inicial entre Python y Node.js responde al tiempo de arranque (cold start) de la máquina virtual V8 en tareas sintéticas pequeñas.

9.2 Actividad 5: Sistema Híbrido "UnegScript" con Distancia Levenshtein y Mock-LLM

Desarrollamos el proyecto **UnegScript Hybrid Parser** para abordar la pérdida de

memoria sintáctica producida por el autocompletado automático ciego en aprendices de ingeniería. El sistema combina un Lexer determinista, la heurística de **Distancia de Levenshtein** (Umbral 0.85) para interceptar errores de mecanografía, y un motor "Mock-LLM" que genera alertas pedagógicas sin alterar el rigor del Parser LL(1).

```
# Código Fuente Entrada con Falencias Mecanográficas:
```

```
whlie x > 0:
```

```
    pront(x)
```

```
    x = x - 1
```

```
retrun True
```

```
# Reporte Final Emitido por el Sistema UnegScript:
```

```
SUGERENCIAS Y ALERTAS FORMATIVAS DE LA IA (Mock-LLM):
```

```
- Línea 1, Col 1: [IA] La palabra clave de iteración es 'while' (Similitud  
Levenshtein: 80%, Confianza IA: 98%)
```

```
- Línea 2, Col 5: [IA] La función estándar de salida es 'print' (Similitud  
Levenshtein: 80%, Confianza IA: 98%)
```

```
- Línea 4, Col 1: [IA] La palabra clave de retorno es 'return' (Similitud  
Levenshtein: 83%, Confianza IA: 98%)
```

```
Estado Final de Construcción de AST: ACEPTADO
```

```
Errores sintácticos fatales: 0
```

10. CONCLUSIÓN

La investigación desarrollada a lo largo de este informe técnico sobre el Análisis Sintáctico permite concluir que esta fase no es meramente un paso intermedio en la compilación, sino el núcleo intelectual que garantiza la coherencia estructural del software moderno. A través del estudio exhaustivo de las gramáticas LL y LR, se ha evidenciado que la elección de una estrategia de parseo es una decisión de ingeniería fundamental que equilibra la facilidad de implementación manual frente a la potencia expresiva necesaria para lenguajes complejos.

Desde la perspectiva teórica, la contraposición entre los métodos descendentes (Top-Down) y ascendentes (Bottom-Up) define las limitaciones y capacidades de los compiladores contemporáneos. Mientras que el análisis LL se destaca por su predictibilidad y claridad diagnóstica —facilitando la detección temprana de errores sintácticos mediante la recursión descendente—, el análisis LR ofrece la robustez necesaria para gramáticas altamente recursivas, consolidándose como el estándar para herramientas de producción a gran escala. La implementación de estas técnicas revela que el dominio de los conjuntos FIRST y FOLLOW no es solo un ejercicio académico, sino la base operativa para cualquier ingeniero que busque optimizar el rendimiento de los analizadores estáticos, linters y motores de refactorización que hoy definen el flujo de trabajo en los Entornos de Desarrollo Integrados (IDEs).

En el plano práctico, el experimento de carga multilingüe realizado por el equipo (Centeno, Longart, Reina y Rodríguez) sobre archivos Docker Compose aportó evidencia empírica sobre la eficiencia de los parsers descendentes recursivos. El benchmarking demostró que, a pesar de las diferencias de rendimiento asociadas a los tiempos de arranque (cold start) de las máquinas virtuales (V8 en Node.js frente a la ejecución en Python), la complejidad algorítmica se mantiene lineal $O(n)$. Este hallazgo valida la viabilidad del paradigma de parseo tradicional incluso en entornos de orquestación modernos, donde la velocidad de

procesamiento de grandes volúmenes de tokens es crítica.

Finalmente, el proyecto "UnegScript" representa una contribución significativa a la pedagogía de la computación. Al integrar la precisión matemática de un parser tradicional con heurísticas de distancia de Levenshtein y asistencia basada en modelos de lenguaje, hemos demostrado que es posible mitigar la atrofia cognitiva causada por el autocompletado ciego. Este enfoque híbrido no solo garantiza el rigor formal exigido por la compilación, sino que transforma al compilador en un tutor activo que guía al estudiante en la corrección de errores mecanográficos, reduciendo la fricción en el aprendizaje de lenguajes formales. En última instancia, esta investigación no solo cierra una etapa académica, sino que abre camino hacia nuevas arquitecturas de compiladores que priorizan tanto la exactitud técnica como la experiencia de aprendizaje del desarrollador.

11. REFERENCIAS BIBLIOGRÁFICAS

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.

Cooper, S. (2023). *Refactorización de código potenciada mediante Árboles de Sintaxis Abstracta*. React Summit 2023.

Grune, D., & Jacobs, C. (2008). *Parsing Techniques: A Practical Guide* (2nd ed.). Springer.

Levine, J. R., Mason, T., & Brown, D. (1992). *lex & yacc* (2nd ed.). O'Reilly Media.

Osorio, C. (2019). *Análisis Sintáctico Ascendente y Autómatas de Pila*. BURGRAM.

Parr, T. (2013). *The Definitive ANTLR 4 Reference* (2nd ed.). Pragmatic Bookshelf.